



AWS EC2 배포 심화

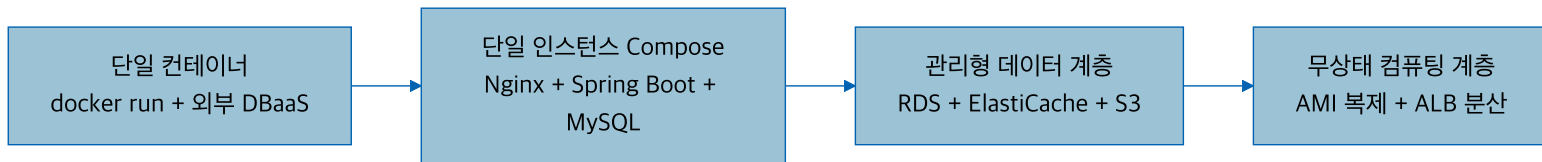
학습 목표

- 데이터 계층을 Amazon RDS, Amazon ElastiCache, Amazon S3로 분리해 EC2를 무상태 컴퓨팅 노드로 전환합니다
- 보안 그룹 체이닝과 비공개 엔드포인트로 데이터 계층의 인터넷 직접 접근을 차단합니다
- 비동기 프로비저닝 요청과 공식 대기자·상태 폴링을 조합하고 Presigned URL로 비공개 S3 객체 접근을 위임합니다
- Compose에서 로컬 MySQL을 제거하고 Spring Boot의 JDBC 연결과 메모리 예산을 RDS 기준으로 재구성합니다
- 커스텀 AMI로 무상태 노드를 복제하고 대상 그룹에 등록해 ALB 라운드로빈 분산을 검증합니다
- 과금이 계속되는 관리형 자원을 실습 당일 전면 삭제해 비용 경계를 관리합니다

상태는 관리형 서비스에 두고 EC2에는 Nginx와 애플리케이션만 남깁니다. 이 분리가 동일 구성의 노드를 복제하고 로드밸런서 뒤에서 수평 확장하는 기반입니다.

핵심 개념 및 구조

인프라 진화: 단일 실행에서 무상태 아키텍처까지



📖 상태 분리

트랜잭션 데이터와 캐시를 EC2 수명 주기에서 분리합니다

🏗️ 수평 확장

동일한 컴퓨팅 노드를 추가해 요청 처리 용량을 확장합니다

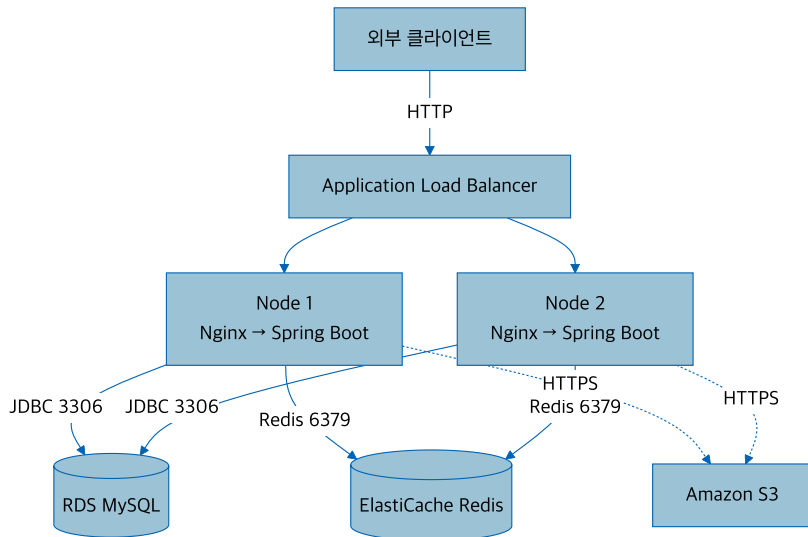
✅ 장애 격리

헬스체크가 비정상 노드를 트래픽 경로에서 제외합니다

관리형 서비스별 책임 분리

구성 요소	주요 역할	격리와 접근 모델
Amazon RDS MySQL	영속 트랜잭션 데이터, 패치와 백업 관리	공인 IP 없이 배치하고 EC2 보안 그룹만 허용 3306
Amazon ElastiCache Redis	인메모리 캐시와 세션 저장소	비공개 엔드포인트에 EC2 보안 그룹만 허용 6379
Amazon S3	정적 파일과 대용량 객체 저장	버킷은 비공개로 유지하고 Presigned URL로 임시 위임
Application Load Balancer	단일 DNS 진입점, 헬스체크, 요청 분산	퍼블릭 서브넷에서 수신하고 정상 EC2의 80 포트로 전달
EC2 Node	Nginx와 Spring Boot 실행	ALB 보안 그룹에서 들어오는 요청만 허용하는 무상태 노드

요청 경로와 비공개 데이터 접근



🛡️ 보안 그룹 체이닝

데이터 포트의 소스를 CIDR이 아닌 EC2 보안 그룹으로 제한합니다

🔗 관리 경로

IntelliJ는 EC2의 **22** 포트를 점프 호스트로 사용해 사설 엔드포인트에 연결합니다

트래픽 처리 계층의 역할 분담

구분	동작 계층	처리 대상	핵심 책임
Application Load Balancer	L7 HTTP	공인 요청과 대상 상태	단일 DNS 진입점, 헬스체크, 다중 EC2 요청 분산
Docker 포트 매핑	L4 TCP NAT	호스트 80 → 컨테이너 80	커널 수준에서 목적지 주소와 포트를 변환
Nginx 리버스 프록시	L7 HTTP	컨테이너 내부 요청	원본 IP 헤더 보존, 백엔드 포트 은닉, app:8080 중계

ALB는 노드 사이에서 요청을 분산하고, Docker는 호스트와 컨테이너 경계를 연결하며, Nginx는 노드 내부에서 HTTP 요청을 애플리케이션으로 중계합니다.

핵심 인프라 개념과 식별 체계

FQDN

전체 도메인 이름 (Fully Qualified Domain Name).
호스트와 도메인 전체 경로를 포함해 IP 변경에도
동일 엔드포인트를 보장합니다.

Presigned URL

사전 서명된 URL. 비공개 S3 객체에 IAM 서명
(SigV4)과 유효 시간을 부여하여 별도 인증 없이
한시적 접근 권한을 위임합니다.

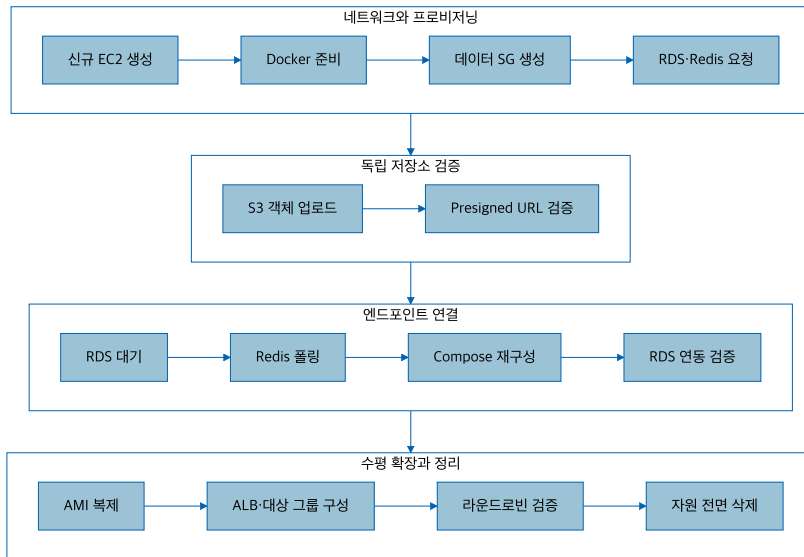
커스텀 AMI

골든 이미지 (Golden Image). 검증된 OS·Docker
런타임·이미지 캐시를 불변 스냅샷으로 굳혀 환경
불일치 없는 고속 복제를 지원합니다.

RDS·ElastiCache·ALB는 FQDN 기반으로 접근을 일원화하고, S3는 Presigned URL로 보안을 통제하며, EC2 수평 확장은 커스텀 AMI를 기반으로 동작합니다.

실습 단계

실습 전체 흐름: 데이터 계층 분리부터 ALB 분산까지



RDS와 ElastiCache 생성 요청을 먼저 접수하고 대기 구간에 S3 작업을 수행합니다. 상태를 관리형 계층으로 옮긴 뒤 AMI 복제본과 ALB로 수평 확장을 검증하고, 과금 자원은 당일 전면 삭제합니다.

실습 식별자와 공통 환경변수

```
export STUDENT_ID="student01"    # 배부받은 사용자명. 바꾸는 곳은 이 출뿐입니다
export GITHUB_USERNAME="your-github-id" # GHCR 이미지 소유자
export AWS_PROFILE="${STUDENT_ID}"
export AWS_REGION="ap-northeast-2"
export AWS_PAGER=""
export MY_KEY_NAME="${STUDENT_ID}-key"
export MY_SG_NAME="${STUDENT_ID}-web-sg"
export MY_INSTANCE_NAME="${STUDENT_ID}-managed-ec2"
export MY_REUSE_INSTANCE_NAME="${STUDENT_ID}-compose-ec2"
export ACTIVE_INSTANCE_NAME="$MY_INSTANCE_NAME"
export MY_APP_IMAGE="ghcr.io/${GITHUB_USERNAME}/simple-back:latest"
```

```
export MY_DATA_SG_NAME="${STUDENT_ID}-data-sg"
export MY_DB_ID="${STUDENT_ID}-mysql-db"
export MY_DB_SUBNET_GROUP="${STUDENT_ID}-db-subnet-group"
export MY_CACHE_ID="${STUDENT_ID}-redis"
export MY_CACHE_SUBNET_GROUP="${STUDENT_ID}-cache-subnet-group"
export MY_AMI_NAME="${STUDENT_ID}-app-image"
export MY_INSTANCE_NAME_2="${MY_INSTANCE_NAME}-2"
export MY_TG_NAME="${STUDENT_ID}-app-tg"
export MY_ALB_SG_NAME="${STUDENT_ID}-alb-sg"
export MY_ALB_NAME="${STUDENT_ID}-app-alb"
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export MY_BUCKET="${STUDENT_ID}-app-assets-${ACCOUNT_ID}"
```

교육용 계정을 여러 수강생이 공유하므로 모든 자원 이름을 `STUDENT_ID` 에서 파생시키고, 생성 시점에 `Name` · `Course=infra-training` · `Owner` 태그를 함께 부여합니다.

AWS Console 결과 확인 경로

결과물	AWS Console 메뉴	검색값과 기대 상태
EC2·보안 그룹	EC2 → Instances / Security Groups	Name= <code>\$MY_INSTANCE_NAME</code> , <code>running</code> ; SG 인바운드 규칙
RDS·ElastiCache	RDS → Databases → Connectivity & security / ElastiCache → Redis caches	<code>\$MY_DB_ID</code> · <code>\$MY_CACHE_ID</code> , <code>Available</code> · 엔드포인트
S3·AMI	S3 → Buckets / EC2 → AMIs	<code>\$MY_BUCKET</code> · <code>\$AMI_ID</code> , 버킷 객체 · <code>available</code>
대상 그룹·ALB	EC2 → Target Groups → Targets / Load Balancers	<code>\$MY_TG_ARN</code> · <code>\$MY_ALB_ARN</code> , <code>healthy</code> · <code>active</code>

Console 검색은 `Name` 태그 또는 리소스 식별자를 사용합니다. Docker 설치·이미지 pull·Compose 로그·API 응답은 AWS Console에 표시되지 않으므로 SSH 터미널에서 `sudo docker` · `sudo docker compose logs` · `curl` 로 확인합니다.

키 페어·웹 보안 그룹 확인

```
cd ~/workspace/aws-lab
export VPC_ID=$(aws ec2 describe-vpcs \
  --filters "Name=is-default,Values=true" --query "Vpcs[0].VpcId" --output text)
export MY_SG_ID=$(aws ec2 describe-security-groups \
  --filters "Name=group-name,Values=$MY_SG_NAME" \
    "Name=vpc-id,Values=$VPC_ID" \
  --query "SecurityGroups[0].GroupId" --output text)
if [ -n "$MY_SG_ID" ] && [ "$MY_SG_ID" != "None" ]; then
  echo "웹 보안 그룹 재사용: $MY_SG_ID"
else
  export MY_SG_ID=$(aws ec2 create-security-group --group-name "$MY_SG_NAME" \
    --description "Web access for managed EC2" --vpc-id "$VPC_ID" \
    --tag-specifications "ResourceType=security-group,Tags=[{Key=Name,Value=$MY_SG_NAME},{Key=Course,Value=infra-training},{Key=Owner,Value=$STUDENT_ID}]" \
    --query "GroupId" --output text)
fi
aws ec2 describe-key-pairs --key-names "$MY_KEY_NAME" --query 'KeyPairs[0].KeyName' --output text
```

기존 키 페어와 보안 그룹은 먼저 조회해 재사용합니다. AWS에 키만 있고 로컬 `./${MY_KEY_NAME}.pem`이 없으면 삭제·재발급하지 말고 강사에게 키 파일을 확인합니다.

신규 EC2 생성과 Docker 준비

```
export BASE_AMI_ID=$(aws ssm get-parameter \
  --name \
  /aws/service/canonical/ubuntu/server/26.04/stable/current/arm64/hvm/ebs-gp3/ami-id \
  --query "Parameter.Value" --output text)
export INSTANCE_ID=$(aws ec2 run-instances \
  --image-id "$BASE_AMI_ID" --instance-type t4g.small \
  --key-name "$MY_KEY_NAME" --security-group-ids "$MY_SG_ID" \
  --tag-specifications \
  "ResourceType=instance,Tags=[{Key=Name,Value=$MY_INSTANCE_NAME},{Key=Course,Value=infra-training},{Key=Owner,Value=$STUDENT_ID}]" \
  --query "Instances[0].InstanceId" --output text)
aws ec2 wait instance-running --instance-ids "$INSTANCE_ID"
aws ec2 wait instance-status-ok --instance-ids "$INSTANCE_ID"
export PUBLIC_IP=$(aws ec2 describe-instances --instance-ids "$INSTANCE_ID" \
  --query "Reservations[0].Instances[0].PublicIpAddress" --output text)
echo "$INSTANCE_ID $PUBLIC_IP"
```

신규 노드

Ubuntu ARM 이미지와 `t4g.small`을 사용해 관리형 서비스 전용 컴퓨팅 노드를 만듭니다. 기존 인스턴스의 상태에 의존하지 않습니다.

Console: EC2 → Instances에서 Name=`\$MY\_INSTANCE\_NAME`을 검색하고 `running`, Status check `2/2 checks passed`를 확인합니다.

EC2 Docker 런타임과 이미지 확인

```
ssh -i ./"$MY_KEY_NAME".pem -o StrictHostKeyChecking=accept-new \  
ubuntu@"$PUBLIC_IP" bash -s <<EOF  
set -e  
curl -fsSL https://get.docker.com -o get-docker.sh  
sudo sh get-docker.sh  
sudo systemctl is-active docker  
sudo docker compose version  
sudo docker pull nginx:alpine  
sudo docker pull "$MY_APP_IMAGE"  
EOF
```

▣ 원격 확인

신규 인스턴스에 Docker 엔진을 설치하고 데몬 상태를 점검한 뒤, Nginx 및 GHCR 애플리케이션 이미지를 사전 다운로드합니다.

Docker 설치·pull 결과는 Console 화면이 아니라 EC2 SSH 터미널의 명령 출력과 `systemctl is-active docker` 로 확인합니다.

선택 절차: 중지 인스턴스 재사용

```
export REUSE_INSTANCE_ID=$(aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=$MY_REUSE_INSTANCE_NAME" \
    "Name=tag:Owner,Values=$STUDENT_ID" \
    "Name=instance-state-name,Values=stopped" \
  --query "Reservations[0].Instances[0].InstanceId" --output text)
if [ "$REUSE_INSTANCE_ID" = "None" ]; then
  echo "재사용할 중지 인스턴스가 없습니다."; exit 1
fi
export INSTANCE_ID="$REUSE_INSTANCE_ID"
export ACTIVE_INSTANCE_NAME="$MY_REUSE_INSTANCE_NAME"
aws ec2 start-instances --instance-ids "$INSTANCE_ID"
aws ec2 wait instance-status-ok --instance-ids "$INSTANCE_ID"
export PUBLIC_IP=$(aws ec2 describe-instances --instance-ids "$INSTANCE_ID" \
  --query "Reservations[0].Instances[0].PublicIpAddress" --output text)

ssh -i ./"$MY_KEY_NAME".pem -o StrictHostKeyChecking=accept-new \
  ubuntu@"$PUBLIC_IP" bash -s <<EOF
set -e
[ -f compose-mysql.yml ] && sudo docker compose -f compose-mysql.yml down || true
sudo rm -rf ./mysql-data compose-mysql.yml .env.mysql
sudo docker pull nginx:alpine && sudo docker pull "$MY_APP_IMAGE"
EOF
```

🔄 선택 재사용

신규 생성 대신 선행 실습에서 중지해 둔 인스턴스를 재가동합니다. 재시작 시 동적 공인 IP가 변경되므로 다시 조회하고, 기존 로컬 MySQL 스택을 정리합니다.

`REUSE_INSTANCE_ID`가 `None`이면 중지 상태 인스턴스가 없는 것이므로 기본 신규 생성 흐름을 진행합니다.

데이터 계층 보안 그룹 생성

```
SUBNET_IDS=$(aws ec2 describe-subnets --filters "Name=vpc-id,Values=$VPC_ID" \
--query "Subnets[].SubnetId" --output text))

export MY_DATA_SG_ID=$(aws ec2 create-security-group \
--group-name "$MY_DATA_SG_NAME" \
--description "RDS and ElastiCache access from EC2 SG" \
--vpc-id "$VPC_ID" \
--tag-specifications "ResourceType=security-group,Tags=[{Key=Name,Value=$MY_DATA_SG_NAME},{Key=Course,Value=infra-training},{Key=Owner,Value=$STUDENT_ID}]" \
--query "GroupId" --output text)

aws ec2 authorize-security-group-ingress \
--group-id "$MY_DATA_SG_ID" --protocol tcp --port 3306 --source-group "$MY_SG_ID"

aws ec2 authorize-security-group-ingress \
--group-id "$MY_DATA_SG_ID" --protocol tcp --port 6379 --source-group "$MY_SG_ID"
```

데이터 포트는 `0.0.0.0/0`이 아니라 EC2 보안 그룹 ID에서 들어오는 연결만 허용합니다.

서브넷 그룹 생성과 Multi-AZ 준비

```
aws rds create-db-subnet-group \  
  --db-subnet-group-name "$MY_DB_SUBNET_GROUP" \  
  --db-subnet-group-description "Default VPC subnets for RDS" \  
  --subnet-ids "${SUBNET_IDS[@]}"  
  
aws elasticache create-cache-subnet-group \  
  --cache-subnet-group-name "$MY_CACHE_SUBNET_GROUP" \  
  --cache-subnet-group-description "Default VPC subnets for cache" \  
  --subnet-ids "${SUBNET_IDS[@]}"
```

다중 AZ 서브넷 필수

RDS와 ElastiCache 서브넷 그룹은 최소 2개 이상의 가용 영역(AZ) 서브넷 등록을 필수로 요구합니다.

Failover 대비

단일 AZ 장애 발생 시 다른 AZ의 Standby 복제본으로 즉시 자동 장애 조치를 수행하기 위한 사전 기반입니다.

실습에서는 단일 노드로 시작하더라도 AWS 관리형 인프라 규격에 따라 다중 가용 영역 서브넷 풀을 먼저 선언합니다.

RDS와 ElastiCache 비동기 프로비저닝

```
read -rsp "RDS master password: " MY_DB_PASSWORD; echo
echo
export MY_DB_PASSWORD

aws rds create-db-instance \
  --db-instance-identifier "$MY_DB_ID" \
  --db-instance-class db.t4g.micro --engine mysql \
  --master-username admin --master-user-password "$MY_DB_PASSWORD" \
  --allocated-storage 20 --storage-type gp3 \
  --vpc-security-group-ids "$MY_DATA_SG_ID" \
  --db-subnet-group-name "$MY_DB_SUBNET_GROUP" \
  --no-multi-az --no-publicly-accessible \
  --backup-retention-period 0 \
  --tags Key=Name,Value="$MY_DB_ID" Key=Course,Value=infra-training Key=Owner,Value="$STUDENT_ID"

aws elasticache create-cache-cluster \
  --cache-cluster-id "$MY_CACHE_ID" \
  --cache-node-type cache.t4g.micro --engine redis \
  --num-cache-nodes 1 \
  --cache-subnet-group-name "$MY_CACHE_SUBNET_GROUP" \
  --security-group-ids "$MY_DATA_SG_ID" \
  --tags Key=Name,Value="$MY_CACHE_ID" Key=Course,Value=infra-training Key=Owner,Value="$STUDENT_ID"
```

두 리소스는 프로비저닝 완료까지 수 분이 소요되므로 동기 대기하지 않고 비동기 호출 후 S3 실습을 병렬로 진행합니다.

S3 버킷 생성과 객체 업로드

```
export ACCOUNT_ID=$(aws sts get-caller-identity \
  --query Account --output text)
export MY_BUCKET="${STUDENT_ID}-app-assets-${ACCOUNT_ID}"

aws s3 mb "s3://$MY_BUCKET" --region "$AWS_REGION"
aws s3api put-bucket-tagging \
  --bucket "$MY_BUCKET" \
  --tagging "TagSet=[{Key=Name,Value=$MY_BUCKET},{Key=Course,Value=infra-training},{Key=Owner,Value=$STUDENT_ID}]"

echo "Hello from Amazon S3 Managed Storage!" > hello.txt
aws s3 cp hello.txt "s3://$MY_BUCKET/hello.txt"
aws s3 ls "s3://$MY_BUCKET/"
```

S3 버킷 이름은 글로벌 네임스페이스에서 유일해야 합니다. AWS 계정 ID를 조합하고 `aws s3 ls` 에서 객체 키와 크기를 확인합니다.

Presigned URL 발급과 다운로드 검증

```
export PRESIGNED_URL=$(aws s3 presign \  
  "s3://$MY_BUCKET/hello.txt" --expires-in 300)  
  
curl -i -s "$PRESIGNED_URL"  
  
echo "$PRESIGNED_URL" | grep -o 'X-Amz-Algorithm=[^&]*'  
echo "$PRESIGNED_URL" | grep -o 'X-Amz-Credential=[^&]*'  
echo "$PRESIGNED_URL" | grep -o 'X-Amz-Expires=[^&]*'  
echo "$PRESIGNED_URL" | grep -o 'X-Amz-Signature=[^&]*'
```

URL 자체가 만료 전까지의 접근 권한입니다. 터미널 기록과 외부 공유를 통제하고, 응답의 **200 OK**와 만료 후 **403 Forbidden**을 대조합니다.

RDS 완료 대기과 엔드포인트 조회

```
aws rds wait db-instance-available \
  --db-instance-identifier "$MY_DB_ID"

export RDS_ENDPOINT=$(aws rds describe-db-instances \
  --db-instance-identifier "$MY_DB_ID" \
  --query "DBInstances[0].Endpoint.Address" --output text)
export RDS_PORT=$(aws rds describe-db-instances \
  --db-instance-identifier "$MY_DB_ID" \
  --query "DBInstances[0].Endpoint.Port" --output text)

echo "$RDS_ENDPOINT:$RDS_PORT"
```

FQDN (전체 도메인 이름) 엔드포인트

RDS 엔드포인트(*.rds.amazonaws.com)는 완전한 도메인 이름(FQDN)으로, 내부 IP 변경에도 클라이언트 애플리케이션의 연결 주소를 영구 고정합니다.

공식 대기자는 상태가 `available` 이 될 때까지 폴링하며, 조회된 FQDN과 `3306` 포트를 다음 단계의 JDBC 연결 정보로 사용합니다.

ElastiCache 상태 폴링과 엔드포인트 조회

```
while [ "$(aws elasticache describe-cache-clusters \
  --cache-cluster-id "$MY_CACHE_ID" \
  --query "CacheClusters[0].CacheClusterStatus" \
  --output text)" ≠ "available" ]; do
  aws elasticache describe-cache-clusters \
    --cache-cluster-id "$MY_CACHE_ID" \
    --query "CacheClusters[0].CacheClusterStatus" --output text
  sleep 15
done

export REDIS_ENDPOINT=$(aws elasticache describe-cache-clusters \
  --cache-cluster-id "$MY_CACHE_ID" --show-cache-node-info \
  --query "CacheClusters[0].CacheNodes[0].Endpoint.Address" \
  --output text)
export REDIS_PORT=$(aws elasticache describe-cache-clusters \
  --cache-cluster-id "$MY_CACHE_ID" --show-cache-node-info \
  --query "CacheClusters[0].CacheNodes[0].Endpoint.Port" \
  --output text)
echo "$REDIS_ENDPOINT:$REDIS_PORT"
```

ElastiCache에는 전용 CLI 대기자가 없으므로 상태 조회 루프를 사용합니다. `--show-cache-node-info`가 실제 노드 엔드포인트를 응답에 포함합니다.

EC2 환경변수를 RDS 연결 정보로 전환

```
ssh -o StrictHostKeyChecking=accept-new -i ./"$MY_KEY_NAME".pem \  
  ubuntu@"$PUBLIC_IP" bash -s <<EOF  
set -e  
cat <<ENVEOF > .env  
DB_NAME=appdb  
DB_USER=admin  
DB_PASSWORD=$MY_DB_PASSWORD  
RDS_ENDPOINT=$RDS_ENDPOINT  
MY_APP_IMAGE=$MY_APP_IMAGE  
APP_MESSAGE=Live on AWS EC2 Node 1 via Compose + RDS!  
SPRING_DATASOURCE_HIKARI_CONNECTIONTIMEOUT=30000  
SPRING_JPA_HIBERNATE_DDL_AUTO=update  
ENVEOF  
EOF
```

인스턴스를 재시작하면 공인 IP가 바뀌므로 `StrictHostKeyChecking=accept-new`로 호스트 키 검증 실패를 방지합니다.
HikariCP 풀은 기동 후 설정이 봉인되므로 타임아웃 키는 `SPRING_DATASOURCE_HIKARI_CONNECTIONTIMEOUT` 하나만
주입합니다.

Compose에서 로컬 DB 제거와 메모리 재배분

```
name: aws-managed
services:
  nginx:
    image: nginx:alpine
    restart: always
    ports: ["80:80"]
    volumes: [".:/nginx.conf:/etc/nginx/nginx.conf:ro"]
    depends_on: [app]
    deploy: { resources: { limits: { memory: 64M } } }
    networks: [frontend-net]
  app:
    image: ${MY_APP_IMAGE}
    restart: on-failure
    env_file: [.env]
    environment:
      PORT: 8080
      JAVA_TOOL_OPTIONS: "-XX:MaxRAMPercentage=75.0"
      SPRING_DATASOURCE_URL: "jdbc:mysql://${RDS_ENDPOINT}:3306/${DB_NAME}?createDatabaseIfNotExist=true"
      SPRING_DATASOURCE_USERNAME: ${DB_USER}
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
    deploy: { resources: { limits: { memory: 1024M } } }
    networks: [frontend-net]
networks: { frontend-net: }
```

외부 상태 계층

로컬 **db** 서비스와 볼륨을 제거하고 JDBC 목적지를 RDS로 변경합니다. 인스턴스 실가용 메모리 약 1.8GiB 기준으로 애플리케이션 상한을 **1024M**으로 잡습니다

프로젝트명을 **name: aws-managed**로 고정하고, 덮어쓰기 전 **sudo docker compose -p aws-3-tier down**으로 이전 스택을 내려 **80** 포트 점유 충돌을 막습니다.

Compose 재기동과 RDS 연동 검증

```
ssh -o StrictHostKeyChecking=accept-new -i ./"$MY_KEY_NAME".pem \  
  ubuntu@"$PUBLIC_IP" bash -s <<'EOF'  
set -e  
sudo docker compose up -d --remove-orphans  
sudo docker compose ps  
sudo docker compose logs --tail 30 app  
EOF  
  
curl -i "http://$PUBLIC_IP/users"
```

`--remove-orphans`가 Compose 정의에서 제거된 로컬 MySQL 컨테이너를 정리합니다. `docker compose ps`의 실행 상태, HikariCP 연결 로그, API의 `200 OK`를 순서대로 확인합니다.

ElastiCache 연결 검증

```
ssh -o StrictHostKeyChecking=accept-new -i ./"$MY_KEY_NAME".pem \  
  ubuntu@"$PUBLIC_IP" bash -s <<EOF  
set -e  
sudo apt-get update -y  
sudo apt-get install -y redis-tools  
redis-cli -h "$REDIS_ENDPOINT" -p 6379 ping  
EOF
```

기대 응답

PING에 **PONG**이 반환되면 EC2에서 비공개
ElastiCache 노드까지 TCP 경로가 정상입니다.

응답이 없으면 데이터 계층 보안 그룹이 EC2 보안
그룹을 소스로 삼아 **6379/tcp**를 허용하는지
확인합니다.

IntelliJ IDEA에서 비공개 RDS 조회

영역	설정	값
MySQL	Host · Port	<code>\$RDS_ENDPOINT</code> · <code>3306</code>
MySQL	인증 · Database	<code>admin</code> · <code>appdb</code>
SSH 터널	Host · Port	<code>\$PUBLIC_IP</code> · <code>22</code>
SSH 터널	사용자 · 키	<code>ubuntu</code> · <code>~/workspace/aws-lab/\${STUDENT_ID}-key.pem</code>

 IntelliJ IDEA의 `Database Tools and SQL`에서 `Use SSH tunnel`을 활성화합니다. 연결 경로는 로컬 → EC2 `22` → RDS `3306`입니다.

```
SELECT DATABASE(); SHOW TABLES; SELECT * FROM users;
```

IntelliJ IDEA에서 비공개 ElastiCache 조회

영역	설정	값
Redis	Host · Port	<code>\$REDIS_ENDPOINT</code> · <code>6379</code>
Redis	인증	실습 클러스터는 <code>No auth</code>
SSH 터널	구성	RDS 연결에서 만든 EC2 SSH 구성 재사용
검증	연결 상태	<code>Test Connection</code> 성공 확인



ElastiCache는 공인 IP가 없으므로 로컬에서 직접 연결하지 않습니다. SSH 터널이 사실 VPC 내부 연결을 대리합니다.

```
SET student01:intellij "connected-via-ssh-tunnel" → GET student01:intellij → DEL student01:intellij
```

커스텀 AMI로 무상태 인스턴스 복제

```
AMI_ID=$(aws ec2 create-image \
  --instance-id "$INSTANCE_ID" \
  --name "${MY_AMI_NAME}-${date +%s}" \
  --tag-specifications \
    "ResourceType=image,Tags=[{Key=Name,Value=${MY_AMI_NAME}},{Key=Course,Value=infra-training},{Key=Owner,Value=${STUDENT_ID}}]" \
  --no-reboot --query ImageId --output text)
aws ec2 wait image-available --image-ids "$AMI_ID"

INSTANCE_ID_2=$(aws ec2 run-instances \
  --image-id "$AMI_ID" --instance-type t4g.small \
  --key-name "$MY_KEY_NAME" \
  --security-group-ids "$MY_SG_ID" \
  --tag-specifications \
    "ResourceType=instance,Tags=[{Key=Name,Value=${MY_INSTANCE_NAME_2}},{Key=Course,Value=infra-training},{Key=Owner,Value=${STUDENT_ID}}]" \
  --user-data '#!/bin/bash
cd /home/ubuntu
sed -i \
  "s/^APP_MESSAGE=.*/APP_MESSAGE=Live on AWS EC2 Node 2 (cloned) via Compose + RDS!/" \
  .env
sudo docker compose up -d' \
  --query 'Instances[0].InstanceId' --output text)
aws ec2 wait instance-status-ok --instance-ids "$INSTANCE_ID_2"
```

골든 이미지

검증된 OS, Docker, 이미지
캐시와 설정을 동일한 불변
상태로 복제합니다.

`user-data`가 `.env`의
`APP_MESSAGE`만 Node 2로
바꾸고 스택을 기동합니다.
이 치환이 있어야 분산
검증에서 두 노드를 구분할
수 있습니다.

대상 그룹 등록과 ALB 보안 그룹 생성

```
MY_TG_ARN=$(aws elbv2 create-target-group \
  --name "$MY_TG_NAME" --protocol HTTP --port 80 \
  --vpc-id "$VPC_ID" \
  --target-type instance \
  --health-check-path / \
  --tags \
    Key=Name,Value="$MY_TG_NAME" \
    Key=Course,Value=infra-training \
    Key=Owner,Value="$STUDENT_ID" \
  --query 'TargetGroups[0].TargetGroupArn' --output text)
aws elbv2 register-targets --target-group-arn "$MY_TG_ARN" \
  --targets "Id=$INSTANCE_ID" "Id=$INSTANCE_ID_2"

MY_ALB_SG_ID=$(aws ec2 create-security-group \
  --group-name "$MY_ALB_SG_NAME" --vpc-id "$VPC_ID" \
  --description "ALB public HTTP endpoint" \
  --tag-specifications \
    "ResourceType=security-group,Tags=[{Key=Name,Value=${MY_ALB_SG_NAME}},{Key=Course,Value=infra-training},{Key=Owner,Value=${STUDENT_ID}}]" \
  --query 'GroupId' --output text)
aws ec2 authorize-security-group-ingress \
  --group-id "$MY_ALB_SG_ID" --protocol tcp --port 80 --cidr 0.0.0.0/0
```

📌 라우팅 기준

대상 그룹 포트는
컨테이너의 **8080**이 아니라
Nginx가 호스트에 노출한
80입니다.

공인 진입점은 ALB 보안
그룹 하나로 모읍니다. EC2
보안 그룹의 인터넷 직접
개방은 다음 단계에서
회수합니다.

ALB 생성과 리스너 연결

```
ALB_SUBNETS=$(aws ec2 describe-subnets \
  --filters "Name=vpc-id,Values=$VPC_ID" \
  --query "Subnets[].SubnetId" --output text))

MY_ALB_ARN=$(aws elbv2 create-load-balancer \
  --name "$MY_ALB_NAME" \
  --subnets "${ALB_SUBNETS[@]}" --security-groups "$MY_ALB_SG_ID" \
  --scheme internet-facing --type application \
  --tags Key=Name,Value="$MY_ALB_NAME" Key=Course,Value=infra-training Key=Owner,Value="$STUDENT_ID" \
  --query 'LoadBalancers[0].LoadBalancerArn' --output text)

aws elbv2 create-listener --load-balancer-arn "$MY_ALB_ARN" \
  --protocol HTTP --port 80 \
  --default-actions "Type=forward,TargetGroupArn=$MY_TG_ARN"

export ALB_DNS=$(aws elbv2 describe-load-balancers \
  --load-balancer-arns "$MY_ALB_ARN" \
  --query 'LoadBalancers[0].DNSName' --output text)
echo "http://$ALB_DNS"
```

서브넷 범위

기본 VPC의 전체 서브넷(가용 영역당 1개)을 지정해 인스턴스가 배치된 가용 영역을 반드시 포함합니다.

일부 서브넷만 지정하면 해당 가용 영역의 대상이 **unused**로 남아 ALB가 **503**만 반환합니다.

ALB 경계 잠금과 분산 검증

```
aws ec2 revoke-security-group-ingress \
  --group-id "$MY_SG_ID" --protocol tcp --port 80 \
  --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress \
  --group-id "$MY_SG_ID" --protocol tcp --port 80 \
  --source-group "$MY_ALB_SG_ID"

aws elbv2 wait target-in-service \
  --target-group-arn "$MY_TG_ARN" \
  --targets "Id=$INSTANCE_ID" "Id=$INSTANCE_ID_2"
for i in {1..6}; do curl -s "http://$ALB_DNS/"; done
curl -i "http://$ALB_DNS/users"
curl -m 3 "http://$PUBLIC_IP/" || echo "직접 접근 차단"
```

하용 경로

EC2의 **80/tcp**는 ALB 보안 그룹에서 시작된 요청만 수신합니다.

반복 응답에서 두 노드가 교대하고 **/users** 결과가 같으면 무상태 분산과 공유 RDS 정합성이 확인됩니다.

관리형 서비스 삭제

```
aws elbv2 delete-load-balancer \  
  --load-balancer-arn "$MY_ALB_ARN"  
aws elbv2 wait load-balancers-deleted \  
  --load-balancer-arns "$MY_ALB_ARN"  
aws elbv2 delete-target-group --target-group-arn "$MY_TG_ARN"  
  
aws rds delete-db-instance \  
  --db-instance-identifier "$MY_DB_ID" \  
  --skip-final-snapshot --delete-automated-backups  
aws elasticache delete-cache-cluster \  
  --cache-cluster-id "$MY_CACHE_ID"  
aws s3 rm "s3://$MY_BUCKET" --recursive  
aws s3 rb "s3://$MY_BUCKET"
```

☐ 삭제 순서

ALB 삭제 완료를 기다린 뒤 대상 그룹을 제거하고, S3는 객체를 비운 뒤 버킷을 삭제합니다.

ElastiCache와 ALB에는 **stop** 상태가 없습니다. 유휴 상태에도 요금이 발생하므로 실습 종료 전에 삭제합니다.

EC2·AMI·스냅샷 정리

```
ssh -o StrictHostKeyChecking=accept-new -i ./"$MY_KEY_NAME".pem \
ubuntu@$PUBLIC_IP" "sudo docker compose down" || true
export CLEANUP_MODE="terminate" # 다음 실습에서 재사용하면 stop으로 변경
if [ "$CLEANUP_MODE" = "stop" ]; then
    aws ec2 stop-instances --instance-ids "$INSTANCE_ID" "$INSTANCE_ID_2"
    aws ec2 wait instance-stopped --instance-ids "$INSTANCE_ID" "$INSTANCE_ID_2"
else
    aws ec2 terminate-instances --instance-ids "$INSTANCE_ID" "$INSTANCE_ID_2"
    aws ec2 wait instance-terminated --instance-ids "$INSTANCE_ID" "$INSTANCE_ID_2"
fi

AMI_SNAPSHOT_ID=$(aws ec2 describe-images --image-ids "$AMI_ID" \
    --query 'Images[0].BlockDeviceMappings[0].Ebs.SnapshotId' \
    --output text)
aws ec2 deregister-image --image-id "$AMI_ID"
aws ec2 delete-snapshot --snapshot-id "$AMI_SNAPSHOT_ID"
aws ec2 revoke-security-group-ingress \
    --group-id "$MY_SG_ID" --protocol tcp --port 80 \
    --source-group "$MY_ALB_SG_ID" || true
aws ec2 authorize-security-group-ingress \
    --group-id "$MY_SG_ID" --protocol tcp --port 80 \
    --cidr 0.0.0.0/0
aws ec2 delete-security-group --group-id "$MY_ALB_SG_ID"
```

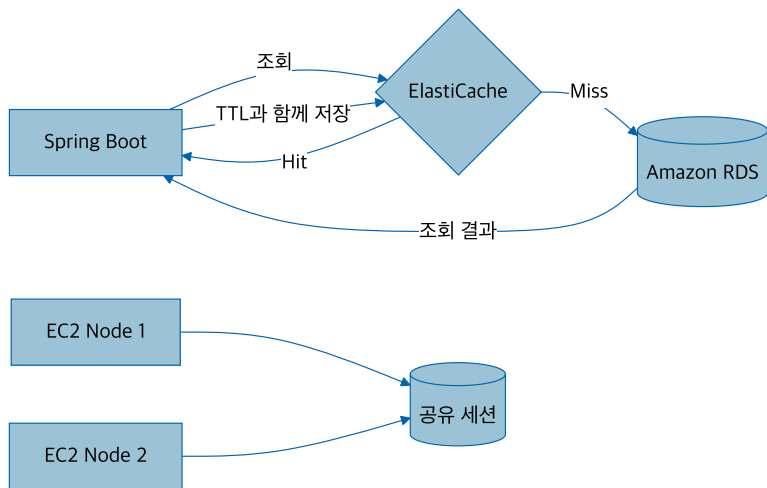
☹ 비용 경계

`CLEANUP_MODE=terminate`는 두 EC2를 종료하고,
`stop`은 다음 실습을 위해 두 대를 보존합니다. AMI
등록 해제만으로는 EBS 스냅샷 요금이 멈추지
않습니다.

Console: EC2 → Instances에서
`\$MY\_INSTANCE\_NAME`·`\$MY\_INSTANCE\_NAME\_2`
의 `terminated` 또는 `stopped` 상태를 확인하고,
AMI·Snapshots 목록에서 `\$MY\_AMI\_NAME` 잔존
여부를 확인합니다.

Appendix

ElastiCache 활용: 캐시와 분산 세션



캐시 어사이드

캐시 미스일 때만 RDS를 조회하고 결과에 TTL을 적용합니다. Spring Boot에서는 `@Cacheable` 로 구성할 수 있습니다.

분산 세션

Spring Session Data Redis로 세션 저장소를 공유하면 어느 노드가 요청을 받아도 로그인 상태가 유지됩니다.

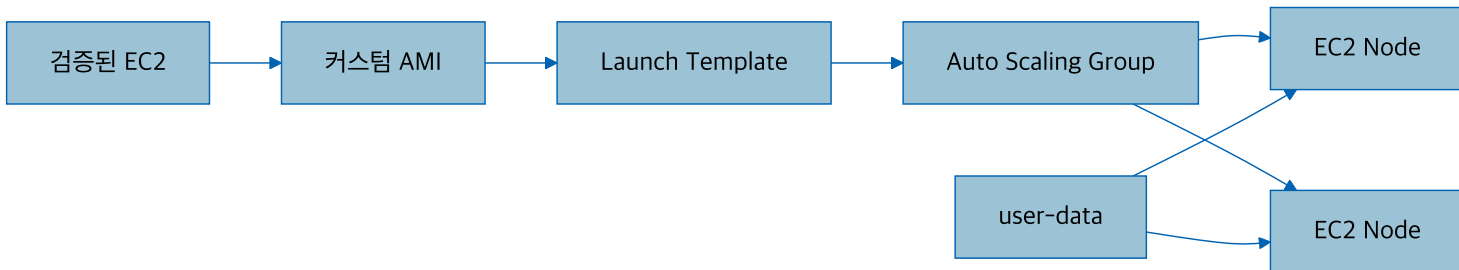
스토리지 선택과 Presigned URL

서비스	접근 모델	데이터 특성
Amazon S3	REST API · 객체 Key	이미지, 아티팩트, 백업 객체
Amazon EBS	OS 블록 디바이스	EC2 부팅 볼륨, 파일시스템
Amazon RDS	SQL · 트랜잭션	관계형 행·열 데이터

S3 Presigned URL은 비공개 버킷의 특정 객체와 HTTP 메서드에 만료 시간을 결합하고, AWS Signature Version 4의 **HMAC-SHA256** 서명을 검증합니다.

URL이 변조되거나 **X-Amz-Expires**가 지나면 S3가 요청을 거부합니다. 버킷 전체 공개보다 최소 권한과 접근 시간 제한을 유지하기 쉽습니다.

골든 이미지와 자동 확장



불변 기반

OS, 런타임, Docker와 이미지 버전을 동일하게 유지합니다.

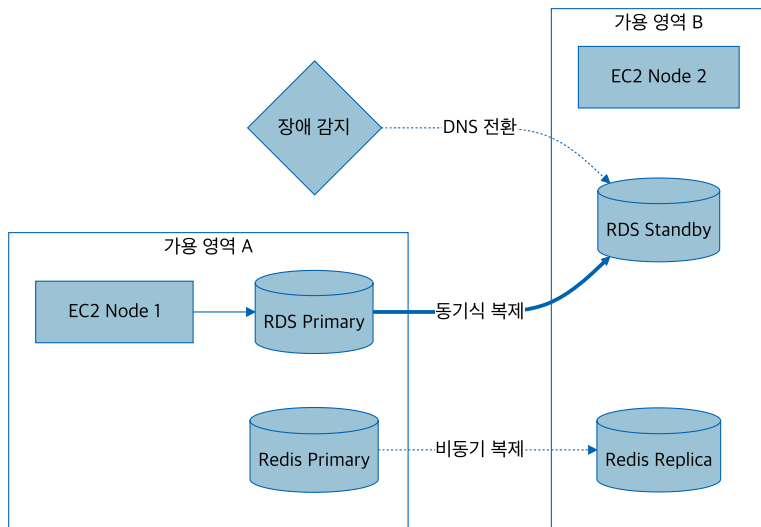
최소 주입

`user-data`는 노드별 설정과 기동만 담당합니다.

신속 복구

수동 SSH 설치 없이 같은 상태의 노드를 반복 생성합니다.

Multi-AZ 장애 조치



RDS Multi-AZ

대기 복제본과 동기식 복제를 유지하고 장애 시 동일 엔드포인트의 대상을 전환합니다.

ElastiCache 복제 그룹

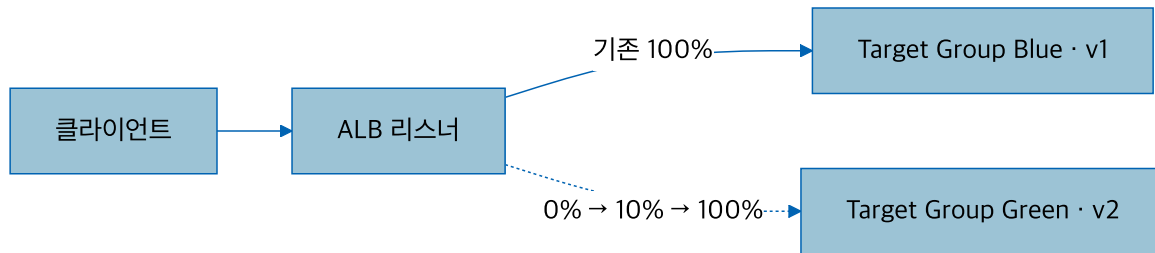
Replica를 다른 AZ에 배치하고 자동 장애 조치 시 새 Primary로 전환합니다.

L4와 L7 로드밸런서 비교

비교 항목	Network Load Balancer (NLB)	Application Load Balancer (ALB)
동작 계층	L4 (전송 계층 / TCP·UDP)	L7 (애플리케이션 계층 / HTTP·HTTPS)
패킷 종단	패킷을 종단하지 않고 대상에 직접 전달	TCP 세션 종단 후 백엔드와 새 연결 수립
라우팅 기준	IP 주소 및 포트 번호 기반 분산	URL 경로, 호스트 헤더, HTTP 메서드 기반
지연 시간	초저지연 (마이크로초 단위, 극단적 처리량)	상대적 지연 (밀리초 단위, L7 헤더 파싱)
주요 사용처	게임 서버, IoT, 금융 등 고속 TCP 트래픽	REST API, MSA 마이크로서비스, 웹 서비스

NLB는 탄력적 IP를 연결한 고정 진입점과 초저지연이 요구될 때 선택하고, ALB는 경로 기반 라우팅과 무종단 Blue/Green 배포가 필요할 때 선택합니다.

ALB Blue/Green 트래픽 제어



본 실습 범위는 단일 대상 그룹의 라운드로빈까지입니다. 복수 대상 그룹의 가중치 전환은 개념 확장 시나리오입니다.

전환	동작	판단 기준
Canary	Green에 일부 요청 전달	헬스체크와 오류율 확인
전체 전환	Green에 모든 요청 전달	이상 시 Blue로 가중치 복원

DNS 조회 흐름과 레코드 유형

레코드 유형	대상 매핑 방식	핵심 특징과 제약
A 레코드	도메인 → IPv4 주소 (<code>192.0.2.1</code>)	고정 IP 직접 매핑, IP 변경 시 TTL 캐시 전파 지연
CNAME 레코드	도메인 → 정규 FQDN (<code>alb-123.amazonaws.com</code>)	정식 별칭 매핑 가능하나 Zone Apex(<code>example.com</code>) 사용 불가
ALIAS 레코드	도메인 → AWS 리소스 (<code>ALB / CloudFront</code>)	Route 53의 가상 A 레코드, Zone Apex 허용, DNS 쿼리 무료

도메인 해석 과정

Local DNS → Root → TLD(.com) → Route 53 Name Server 순으로 질의하여 최종 IP를 획득합니다.

TTL과 캐싱 트레이드오프

TTL이 길면 DNS 조회 부하가 감소하지만 레코드 변경 반영이 느려지고, 짧으면 빠른 전환이 가능하나 쿼리 비용이 증가합니다.

RDS 엔진 선택지와 Amazon Aurora

엔진	호환성 및 라이선스	스토리지 및 복제 특성	적합한 워크로드
MySQL	오픈소스 (GPL)	범용 RDBMS, 읽기 복제본 최대 5개	웹 서비스, 일반 CRUD 애플리케이션
PostgreSQL	오픈소스 (PostgreSQL)	복잡한 쿼리와 지리 정보(GIS) 지원	데이터 분석, 위치 기반 서비스
MariaDB	오픈소스 (LGPL/GPL)	MySQL 드롭인 대체, 다양한 스토리지 엔진	오픈소스 라이선스 거버넌스 환경
Amazon Aurora	AWS 클라우드 네이티브	컴퓨팅-스토리지 분리, 복제본 최대 15개	대규모 트래픽, 초고가용성 시스템

표준 RDBMS는 컴퓨팅과 EBS가 단일 인스턴스에 결합되어 있으나, Aurora는 스토리지를 3개 AZ 분산 계층으로 완전 분리합니다.

RDS 엔진 선택지와 Amazon Aurora

📄 컴퓨팅·스토리지 분리

독립된 분산 스토리지 볼륨을 공유하므로 DB 인스턴스 크래시 시에도 데이터 손실 없이 즉시 복구됩니다.

📄 6방향 복제 & 15개 Replica

3개 AZ에 걸쳐 6벌의 데이터 복제본을 유지하며, 복제 지연 없이 최대 15개의 읽기 전용 복제본으로 확장합니다.

↺ 초고속 자동 Failover

스토리지 재동기화 과정 없이 30초 이내에 읽기 복제본을 새로운 Primary 노드로 자동 전환하여 무중단을 보장합니다.

Amazon Aurora는 MySQL 및 PostgreSQL과 완전 호환되면서도 AWS 클라우드 네이티브 분산 스토리지 아키텍처를 통해 기존 오픈소스 엔진 대비 최대 5배의 처리 성능을 제공합니다.

비용 라이프사이클과 SSH 점프 호스트

리소스	보존 상태	완전 정리
EC2	<code>stop</code> 시 컴퓨팅 요금 중단, EBS 요금 유지	<code>terminate</code>
커스텀 AMI	등록 자체보다 EBS 스냅샷에 과금	AMI 등록 해제 후 스냅샷 삭제
Amazon RDS	일시 중지 중에도 스토리지 요금 유지	DB 인스턴스 삭제
ElastiCache · ALB	중지 상태 없음	각 리소스 삭제

SSH 로컬 포트 포워딩은 로컬 스트림을 EC2의 암호화 세션으로 전달합니다. EC2가 사설 VPC에서 `RDS:3306` 또는 `ElastiCache:6379` 연결을 대리하므로 데이터 계층을 인터넷에 공개하지 않습니다.

핵심 기술 면접 질문

RDS 사설 연결

`--no-publicly-accessible`에서도 EC2 연결이 가능한 이유는 무엇인가?

보안 그룹 체이닝

CIDR 대신 보안 그룹을 소스로 지정하면 무엇이 개선되는가?

ElastiCache 수명주기

RDS와 달리 `stop`이 없는 이유는 무엇인가?

S3 접근 제어

버킷 공개보다 Presigned URL이 안전한 이유는 무엇인가?

핵심 기술 면접 질문

ALB 헬스체크

비정상 대상을 어떻게 감지하고 트래픽에서 격리하는가?

AMI 일관성

`--no-reboot`의 장점과 위험은 무엇인가?

자동 복제

AMI와 `user-data`가 수동 설치보다 나은 이유는 무엇인가?

요청 경로

ALB, Docker 포트 매핑, Nginx의 역할은 어떻게 구분되는가?

핵심 기술 면접 질문

L4 vs L7 선택 기준

NLB와 ALB의 핵심 차이와 상황별 선택 기준은 무엇인가?

Route 53 ALIAS 레코드

CNAME 대비 ALIAS 레코드가 제공하는 기술적·비용적 이점은 무엇인가?

다중 AZ 서브넷 그룹

서브넷 그룹 생성 시 최소 2개 이상의 AZ 서브넷이 필수인 이유는 무엇인가?

Amazon Aurora 아키텍처

기존 RDS 대비 Aurora의 컴퓨팅·스토리지 분리가 주는 성능상 이점은 무엇인가?

모범 답변

RDS 사설 연결

`--no-publicly-accessible` 은 공인 IP와 인터넷 직접 경로를 제거합니다. 동일 VPC의 EC2는 사설 주소와 보안 그룹의 `3306/tcp` 허용 규칙을 통해 연결합니다.

보안 그룹 체이닝

소스 보안 그룹에 속한 인스턴스는 사설 IP와 수량이 바뀌어도 허용 대상이 됩니다. Auto Scaling 환경에서 CIDR 규칙 갱신을 줄이고 역할 기반 경계를 유지합니다.

ElastiCache 수명주기

인메모리 노드는 컴퓨팅 중지과 데이터 보존을 분리하기 어렵습니다. ElastiCache는 활성 클러스터를 유지하거나 삭제하며, 유허 클러스터에도 노드 요금이 발생합니다.

모범 답변

S3 접근 제어

Presigned URL은 비공개 버킷에서 특정 객체와 메서드에만 서명된 임시 권한을 위임합니다. 만료와 서명 검증으로 영구 공개 범위를 만들지 않습니다.

ALB 헬스체크

ALB는 대상 그룹의 경로를 주기적으로 호출합니다. 실패 임계치에 도달한 대상을 분배 풀에서 제외하고, 정상 응답이 회복되면 다시 편입합니다.

AMI 일관성

`--no-reboot`는 서비스 중단을 피하지만 메모리 버퍼의 미기록 데이터가 스냅샷에 빠질 수 있습니다. 로컬 쓰기가 없는 무상태 서버에서 위험이 낮습니다.

모범 답변

AMI와 user-data

검증된 OS와 런타임을 불변 이미지로 복제하고 부팅 시 노드별 설정만 주입합니다. 수동 설치의 버전 편차와 작업 오류를 줄이며 Launch Template과 Auto Scaling Group의 기반이 됩니다.

요청 경로

ALB는 L7 단일 진입점과 헬스 기반 분산을 담당합니다. Docker 포트 매핑은 호스트 `80`을 Nginx 컨테이너 `80`으로 L4 NAT 전달하고, Nginx는 HTTP를 종단해 내부 `app:8080`으로 역방향 프록시합니다.

L4 vs L7 선택 기준

초저지연, 고정 IP, 대규모 TCP/UDP 트래픽 처리는 L4 NLB를 선택하고, URL 경로 기반 라우팅, HTTP 헤더 조작, 무중단 Blue/Green 전환이 필요한 웹/API 환경은 L7 ALB를 선택합니다.

모범 답변

Route 53 ALIAS 레코드

표준 CNAME은 루트 도메인(Zone Apex, `example.com`)에 적용할 수 없지만, ALIAS는 가상 A 레코드로서 Zone Apex를 ALB DNS에 직접 매핑할 수 있으며 AWS 내부 쿼리 비용이 무료입니다.

다중 AZ 서브넷 그룹

단일 가용 영역 장애나 데이터센터 정전 시 Standby 인스턴스로 자동 장애 조치(Failover)를 수행할 수 있도록, 프로비저닝 시점부터 물리적으로 분리된 2개 이상의 AZ 서브넷 풀을 필수로 요구합니다.

Amazon Aurora 아키텍처

컴퓨팅 계층과 스토리지 계층을 분리하고 3개 AZ에 걸쳐 6벌 복제되는 공유 분산 스토리지를 사용합니다. 복제 지연이 거의 없고 최대 15개 Replica 증설 및 30초 이내 고속 Failover를 지원합니다.

깊이 알아보기

Amazon RDS의 VPC 구성

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_VPC.WorkingWithRDSInstanceinaVPC.html

ElastiCache 접근 패턴

<https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/accessing-elasticache.html>

S3 Presigned URL

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html>

Amazon EBS 기반 AMI 생성

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/creating-an-ami-ebs.html>

깊이 알아보기

Elastic Load Balancing 개요

<https://docs.aws.amazon.com/elasticloadbalancing/latest/userguide/what-is-load-balancing.html>

ALB 대상 그룹 헬스체크

<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-health-checks.html>

Route 53 리소스 레코드 유형

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resource-record-types.html>

Amazon Aurora 아키텍처 개요

https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/CHAP_AuroraOverview.html



- EOF -